Chapter 6. Subwording and Transformers

Machine Translation: Advanced Topics

Vincent Vandeghinste

KU Leuven – Brussels

2026

Subwording in Neural Machine Translation

Why subwording? (word-level weaknesses)

Early NMT often used **word-level vocabularies**, but this creates problems:

- **OOV problem**: unseen words become `<unk>` (names, terminology, inflections, compounds).
- **Data sparsity**: related forms are independent symbols (*run*, *runs*, *running*, *ran*).
- **Large vocabularies** increase model size and training cost (bigger embeddings + output layers).

Subwording addresses all three simultaneously.

Core idea of subwording

Instead of treating the **word** as the smallest unit, we use **subword units** learned from data.

- More flexible than words: can compose unseen forms.
- More structured than characters: sequences stay manageable.

Example segmentation:

un + believ + able

Benefits:

- smaller vocabulary,
- no <unk> (everything can be composed),
- morphology is learned implicitly via recurring pieces.

Byte Pair Encoding (BPE): idea

BPE starts from **characters** and repeatedly merges the most frequent adjacent pair:

1. find most frequent adjacent symbol pair,
2. merge into a new symbol,
3. repeat for a fixed number of merge operations.

Resulting vocabulary contains:

- characters,
- frequent character sequences,
- full words where appropriate.

BPE: tiny illustrative segmentation

You can think of BPE as gradually building longer units:

$$u\ n\ b\ e\ l\ i\ e\ v\ a\ b\ l\ e \Rightarrow un + believ + able$$

In practice, merges are learned from corpus frequency statistics.

BPE: Reminder of the algorithm

Starting point: corpus split into characters + end-of-word symbol `</w>`.

Iterate:

1. Count all adjacent symbol pairs.
2. Select the most frequent pair.
3. Merge it into a new symbol.
4. Update corpus and repeat.

We stop after a fixed number of merges.

BPE for MT: Important detail about </w>

In subword BPE for MT (Sennrich et al., 2016):

- The end-of-word marker </w> is **attached to the final character**.
- It is **not a free-standing symbol**.
- Therefore:

$$\texttt{w} \neq \texttt{w}\texttt{</w>}$$

This guarantees:

- No merges cross word boundaries.
- Word-final characters are distinguished from word-internal ones.

Toy corpus

Assume the tiny corpus:

low
lowest
newer
wider

Character-level representation with EOW attached:

l o w</w>
l o w e s t</w>
n e w e r</w>
w i d e r</w>

Initial vocabulary = all individual symbols.

Step 1: Count adjacent symbol pairs

Pairs now include:

- (l, o)
- (o, w</w>) ← from `low`
- (o, w) ← from `lowest`
- (w, e)
- (e, r</w>)
- (e, r)

Notice:

`(o, w</w>)` and `(o, w)` are different pairs.

Step 1: Most frequent pair

Counting frequencies across the corpus:

Pair	Frequency
(l, o)	2
(w, e)	2
(e, r</w>)	2

Assume we select:

(l, o)

Step 1: Merge (l, o) → lo

Updated corpus:

```
lo w</w>
lo w e s t</w>
n e w e r</w>
w i d e r</w>
```

New symbol added: `lo`

Step 2: Recount pairs

Now the frequent pairs include:

- (lo, w</w>)
- (lo, w)
- (w, e)
- (e, r</w>)

Frequencies:

Pair	Frequency
(lo, w</w>)	1
(lo, w)	1
(w, e)	2
(e, r</w>)	2

Assume we select:

(w, e)

Step 2: Merge (w, e) → we

Updated corpus:

```
lo w</w>
lo we s t</w>
n e we r</w>
w i d e r</w>
```

New symbol added: `we`

Step 3: Recount again

Frequent pairs now include:

- (e, r</w>)
- (we, r</w>)

Assume we select:
(e, r</w>)

Step 3: Merge (e, r</w>) → er</w>

Updated corpus:

lo w</w>
lo we s t</w>
n e we r</w>
w i d er</w>

Now er</w> is a reusable suffix unit.

After several merges...

Eventually we may obtain units such as:

- low</w>
- est</w>
- new
- er</w>

Example segmentation:

low</w>
low + est</w>
new + er</w>
wid + er</w>

BPE has discovered frequent reusable pieces.

Key Takeaways

- Word-final symbols are distinct (e.g. `w` vs `w</w>`).
- Merges never cross word boundaries.
- BPE is greedy and frequency-driven.
- It approximates morphology — but does not explicitly model it.

SentencePiece Unigram: how it differs from BPE

BPE builds pieces via *deterministic merges*.
Unigram starts with *many candidate pieces* and **prunes** the least useful ones.

Key idea: segmentation is **probabilistic**.

- multiple segmentations are considered during training,
- pieces get probabilities (usefulness),
- low-utility pieces are removed until target vocab size is reached.

SentencePiece corpus representation (whitespace as symbol)

SentencePiece does not rely on external tokenization.

Whitespace becomes a special boundary marker (often shown as _).

Example:

Resumption of the session

Internal representation:

_Resumption _of _the _session

This allows word-initial pieces like `_the`, `_session`.

Initial inventory and competing segmentations

The initial inventory includes:

- all characters (incl. _),
- frequent words (`_the`, `_of`, `_session`),
- frequent fragments/suffixes (`ion`, `tion`, `_sess`, `ss`, ...).

A single word can have multiple candidate segmentations, e.g.:

_session *or* *_sess ion* *or* *_se ss ion*

Learning and pruning intuition

Pieces get probabilities based on how useful they are for explaining the corpus.

- High-probability pieces frequently appear in good segmentations.
- Low-probability pieces rarely help, and become pruning candidates.

Training iterates:

1. estimate piece probabilities,
2. prune least useful pieces,
3. re-estimate on the reduced inventory,
4. repeat until target vocab size.

Example: likelihood of segmentations

Given probabilities (toy example):

Piece	Prob.
▁session	0.30
▁sess	0.10
ion	0.25
▁se	0.05
ss	0.05

Unigram probability of a segmentation is a **product**:
P(▁session)=0.30,
P(▁sess ion)=$0.10 \cdot 0.25 = 0.025$,
P(▁se ss ion)=$0.05 \cdot 0.05 \cdot 0.25 = 0.000625$
Whole-word piece wins here; low-utility pieces (e.g. ▁se, ss) are candidates for pruning.

After training: final segmentation

After convergence, SentencePiece selects the **most likely** piece sequence.
Plausible outcomes:

- frequent function words stay intact: `_the`, `_of`
- frequent words may remain whole: `_session`
- rarer/longer words are decomposed: `_requested` → `_request ed`

Hands-on notebooks (Kaggle)

Subwording + RNN with SentencePiece:

mtat26-subwording-transformers

(Installation commands, training commands, and plotting are documented inside the notebooks.)

Subwording experiments: what we tested

We compare:

- **Separate SentencePiece models** (src and tgt each trained on its own corpus)
- **Joint vocabulary** (single SentencePiece model trained on concatenated corpora)

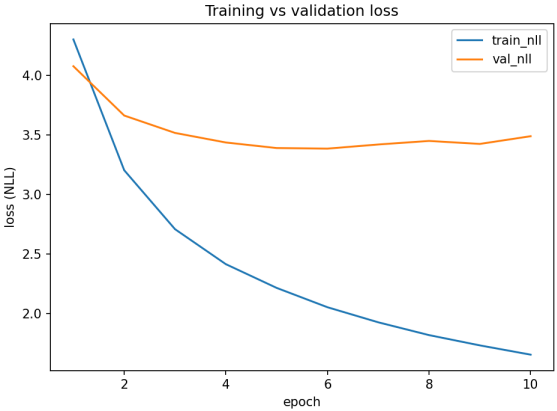
We monitor:

- training vs validation loss (overfitting),
- BLEU and chrF on dev,
- comparison with earlier RNN/GRU/LSTM/biLSTM(+att) baselines.

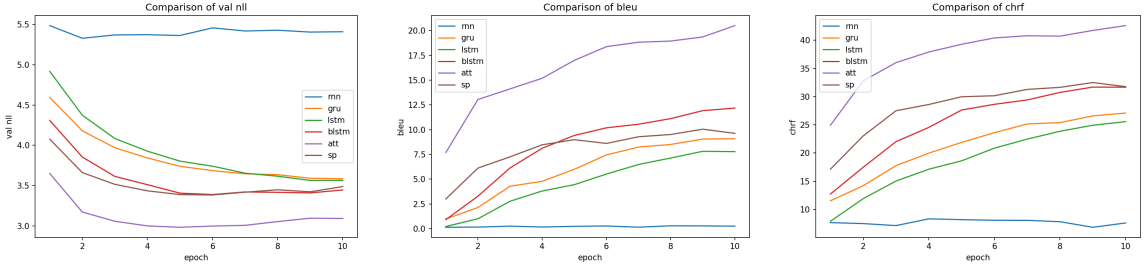
Results: separate SentencePiece models

Key observation from the notes:

- The model still tends to **overfit** after a few epochs.
- In this small-data setting, subwording does **not necessarily improve** BLEU/chrF.



Comparing systems (NLL / BLEU / chrF)



Comparison with earlier RNN-based systems

Joint vocabulary: why it often helps

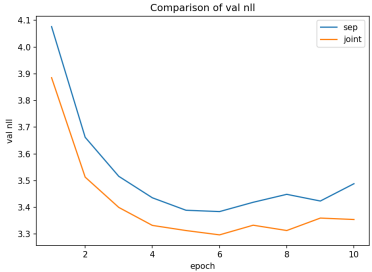
Joint subword vocabularies are common in MT and multilingual settings:

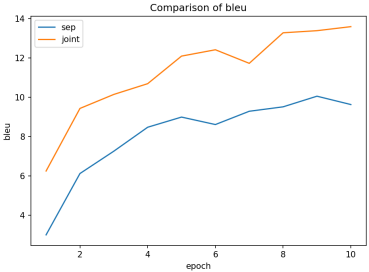
- encourages sharing of subword units across languages,
- helps align cognates/morphemes and shared function words,
- reduces total vocabulary size (one model instead of two).

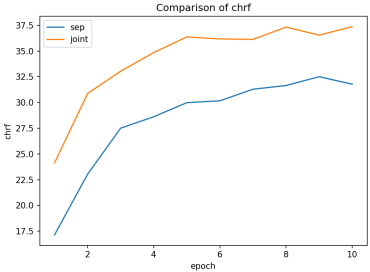
Workflow:

1. concatenate corpora,
2. train SentencePiece on the joint corpus,
3. train NMT with the *same* model for src and tgt.

Results: separate vs joint SentencePiece







Joint SentencePiece vs separate: joint tends to score consistently better in the notes.

Transformers

Motivation: beyond recurrent models

RNNs process tokens **sequentially**:

- limited parallelisation $\Rightarrow$ slower training,
- long-range dependencies can be difficult to capture.

Transformers ((Vaswani et al. 2017)) remove recurrence:

- all tokens processed **in parallel** via attention,
- better scalability and strong MT quality improvements.

Core idea: attention is all you need

Each token directly attends to all other tokens in the sentence, independently of distance.

This is **self-attention**: tokens build contextual representations by selectively focusing on other tokens.

Self-attention: what it does

Given a sentence (after subwording + embeddings + positional encoding), self-attention updates each token representation using information from:

- itself,
- all other tokens in the same sentence.

Example sentence:

She eats apples .

Queries, keys, values (Q, K, V)

For each token we derive:

- **Query** (what I am looking for),
- **Key** (what I offer / how I can be matched),
- **Value** (what information I contribute).

For token `eats`:

- compare $Q_{\texttt{eats}}$ with K of all tokens (`she`, `eats`, `apples`, punctuation),
- obtain attention weights,
- form a weighted sum of the corresponding V vectors.

Attention weights → contextual representations

Self-attention produces **contextualised** vectors:

- `eats` can incorporate information about its subject/object,
- `apples` can attend to `eats` to encode its role as object,
- punctuation typically receives low attention in content heads.

So the representation of a token depends on **the whole sentence**.

Why self-attention matters in MT: pronouns

Source language ambiguity can force explicit choices in the target language.

English pronoun `it` can be ambiguous, but French requires grammatical gender:

- *il* (masc.) vs *elle* (fem.)

Self-attention can connect `it` to the most plausible antecedent in context.

Example from Transformer explanations (in notes)

The animal didn't cross the street because it was too tired.
L'animal n'a pas traversé la rue parce qu'il était trop fatigué.

The animal didn't cross the street because it was too wide.
L'animal n'a pas traversé la rue parce qu'elle était trop large.

Figure: Two sentences requiring pronoun resolution for correct French translation.

Self-attention visualisation for `it`

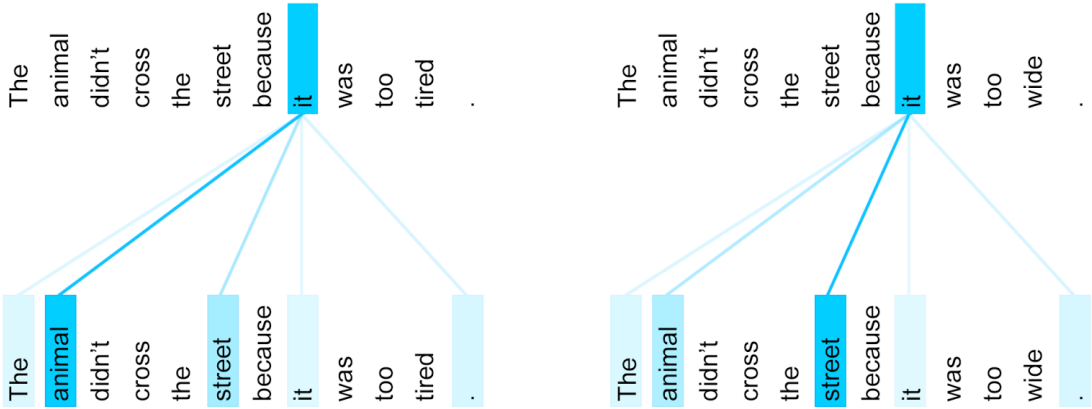


Figure: Attention patterns: `it` attends to different antecedents depending on context.

Multi-head attention

Transformers compute multiple attention distributions in parallel:

- each head has its own Q/K/V projections,
- heads can specialise (agreement, modifiers, long-distance links, ...),
- head outputs are concatenated and linearly transformed.

Breadth: multiple relational views at the same layer.

Multiple layers: progressive refinement

Encoders/decoders are stacks of identical layers.

- lower layers capture more surface/local associations,
- higher layers integrate broader context and semantics,
- ambiguity can be resolved progressively across layers.

Depth: repeated transformation and integration.

Heads + layers: complementary roles

- **Multi-head attention** increases *breadth*: diverse relations in parallel.
- **Stacked layers** increase *depth*: refine and combine evidence over layers.

Both are essential for strong contextual modelling in MT.

Transformer decoder: 3 components per layer

Each decoder layer contains:

1. **Masked self-attention** (only attend to previous target tokens),
2. **Cross-attention** (attend to encoder outputs),
3. **Feed-forward network** (transform combined information).

Stacking decoder layers integrates target context with source context progressively.

Autoregressive decoding (and beam search)

At inference time, Transformers generate tokens **sequentially**:

- each next token conditions on all previous target tokens (via masked self-attention),
- and on the full source sentence (via cross-attention).

Unlike RNNs, there is no single recurrent hidden state; attention recomputes over the available context each step.

Beam search explores multiple hypotheses during decoding.

Why Transformers work well for MT

Transformers:

- avoid recurrence bottlenecks,
- enable parallel training,
- model long-distance dependencies effectively,
- work naturally with subword vocabularies,
- scale well and deliver strong MT quality.

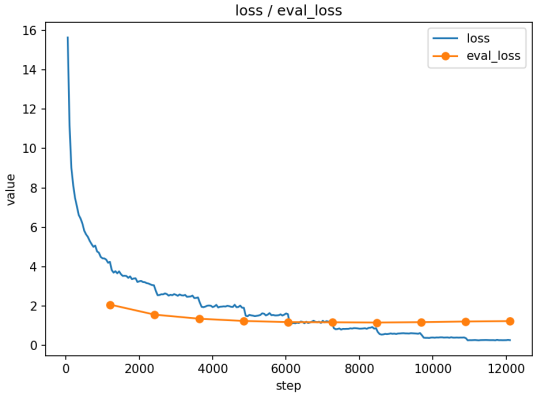
They are now the dominant MT architecture and underpin many pretrained models.

Transformer training: what to look for

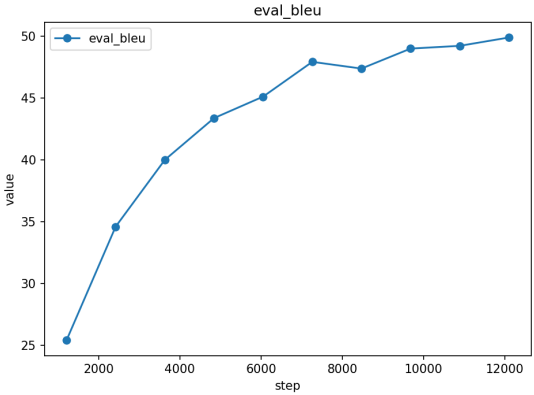
In the hands-on notebook:

- **Transformers (bilingual):** mtat26-transformers
- **Multilingual Transformers + zero-shot:** mtat26-multilingualtransformers
- validation is computed every fixed number of steps (not only per epoch),
- early epochs can show higher train loss than dev loss,
- BLEU can reach much higher values than in the earlier RNN experiments.

Transformer results



Validation loss over epochs.



BLEU score over epochs.

Multilingual Neural Machine Translation

Why multilingual MT?

Training one bilingual model per language pair is often impractical.
MNMT trains **one model** on multiple directions:

- parameter sharing across languages,
- shared subword vocabularies,
- explicit conditioning on the target language.

It raises questions about **cross-lingual transfer** vs **negative transfer**.

Target language tags

A common method: prepend a target-language tag to the source sentence, e.g.

<2nl> She eats apples .

The tag conditions the decoder on the intended output language, enabling one model to serve multiple directions.

Shared subwords in multilingual MT

Shared vocabularies:

- encourage alignment of cognates/morphemes/function words,
- reduce model complexity (one vocab),
- support transfer (especially for related languages).

Zero-shot translation

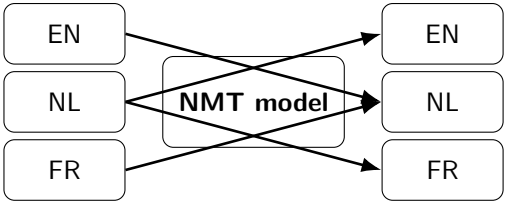
Zero-shot can emerge when a multilingual model learns a shared space:

- trained on some directions (e.g. EN↔NL and FR↔NL),
- can sometimes translate EN↔FR without direct EN–FR supervision.

This relies on **parameter sharing** and **cross-lingual generalisation**.

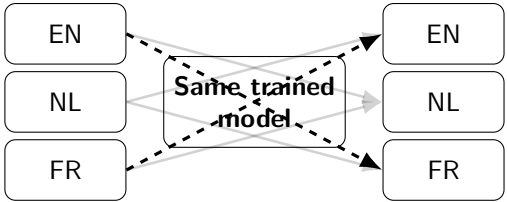
Zero-shot translation in MNMT (training vs inference)

Phase 1: multilingual training



(a) Training directions.

Phase 2: zero-shot inference



(b) Add EN↔FR zero-shot.

Model capacity and data imbalance

Challenges:

- high-resource languages dominate training,
- low-resource directions may degrade,
- interference can cause **negative transfer**.

Mitigations:

- temperature-based sampling / upsampling,
- language-specific adapters / routing,
- increase model capacity.

Evaluation in multilingual MT

Evaluation is more complex than bilingual MT:

- performance varies by language and direction,
- improvements in one pair can coincide with drops in another,
- report per-direction + macro-averages where appropriate.

Hands-on MNMT: pipeline overview

In the multilingual notebook we:

1. add target-language tags to source sentences,
2. concatenate all directions into one multilingual training/dev set,
3. train a **joint** SentencePiece model,
4. train a multilingual Transformer,
5. test a **zero-shot** direction by using a new tag at inference.

Multilingual Transformer results

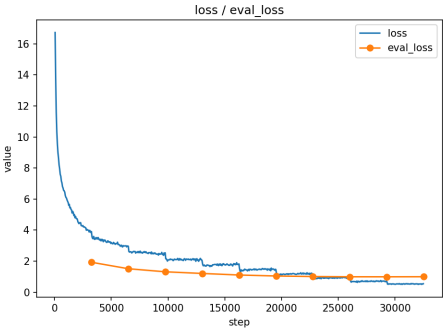


Figure: Validation loss over epochs.

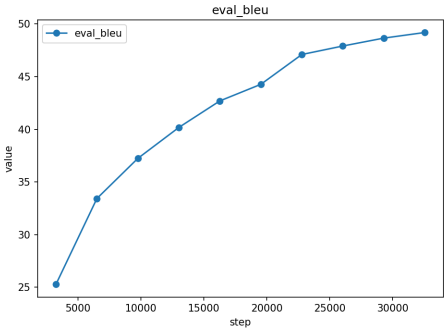


Figure: BLEU score over epochs (combined dev set).

Note: BLEU here aggregates multiple directions, so it is not strictly comparable to bilingual BLEU.

Zero-shot test: what to expect

When testing EN→FR zero-shot on this small setup:

- translations are typically **not very good**,
- but you may observe **some** learned ability to produce French,
- the limiting factor is the **small scale** of the Tatoeba data and limited multilingual signal.

Takeaway: zero-shot is possible in principle, but quality strongly depends on data scale, coverage, and model capacity.